

Mid Topic - CSE-207

Binary search, Ternary search, stack, Queue, Linked List, Recursion.

1. How do we represent LinkedList in memory?
2. Linked List vs 1D Array + Advantages, Disadvantages
3. How to do Insertion & Deletion in Linked List.
4. Properties of Recursive Function. (3/4 marks)
5. Use of Stack.
6. Infix to postfix convert.
7. There will be questions from the rear and front of the queue.

Sample Question

1. Find the postfix equivalent of the following expression and evaluate the result.
 $(4+8)*(7-5)/((4-2)*(2+2))$
2. Suppose you have a circular queue of size 6 (1 to 6). Initially the value of the front and rear is 0. Now show the state of the queue if the following operation is performed on the queue. Also find the value of front and rear in each step. insert (10); insert (5), delete (), insert (9); insert (9); insert (7); insert (5); insert (8); insert (10); insert (11).
3. State the memory representation of single linked list with example.
4. Write a simple procedure to count the number of elements in a linked list.
5. Linked lists have several advantages over arrays. Mention some advantages.
6. State recursive solution for the tower Hanoi problem if the number of discs $N=4$.
7. Define stack. Write down the basic steps for Push operation. Or write some application of stack.
8. Convert the following expression to postfix Q: $((A+B)*D)^{(E-F)}$.
9. Evaluate the following Postfix expression. $P=45,7,2,-,/,2,1,9,+,*,+$
10. Convert this to equivalent postfix expression using stack: $K+L-M*N+(O^P)*W/U/V*T+Q$
11. For the following set of data use binary search to find the value of a number called ITEM=110. $A[11]=\{10,15,45,56,67,70,80,90,100,110,120\}$

1. Find the postfix and evaluate result
 $(4+8)*(7-5)1((4-2)*(2+2))$

Infix	Stack	Postfix
	(	
(	((	
4	((	4
+	((+	4
8	((+	4, 8
)	(	4, 8, +
*	(*	4, 8, +
(	(* (	4, 8, +
7	(* (	4, 8, +, 7
-	(* (-	4, 8, +, 7,
5	(* (-	4, 8, +, 7, 5
)	(*	4, 8, +, 7, 5, -
1	(1	4, 8, +, 7, 5, -, *
(	((1	4
(	((((	4
4	((((	4, 8, +, 7, 5, -, *, 4
-	((((-	4
2	((((-	4, 8, +, 7, 5, -, *, 4, 2
)	(((-	4, 8, +, 7, 5, -, *, 4, 2, -
*	(((*	4
(	(((* (	4
2	(((*	4, 8, +, 7, 5, -, *, 4, 2, 5, 2
+	(((* (+	4
2	(((*	4, 8, +, 7, 5, -, *, 4, 2, -, 2, 2
)	(((*	4, 8, +, 7, 5, -, *, 4, 2, -, 2, 2, +
)	((	4, 8, +, 7, 5, -, *, 4, 2, -, 2, 2, +, *
)	(	4, 8, +, 7, 5, -, *, 4, 2, -, 2, 2, +, *, 1

Convert Infix to postfix

Postfix to result

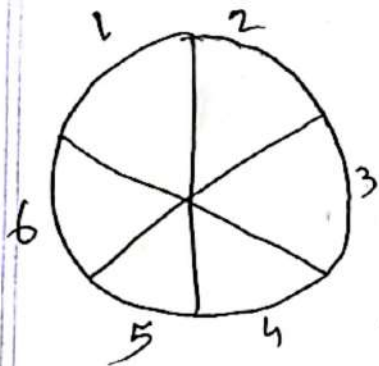
Postfix 4, 8, +, 7, 5, -, *, 4, 2, -, 2, 2, +, *, /

Postfix	Stack
4	4
8	4, 8
+	12
7	12, 7
5	12, 7, 5
-	12, 2
*	24
4	24, 4
2	24, 4, 2
-	24, 2
2	24, 2, 2
2	24, 2, 2, 2
+	24, 2, 4
*	24, 8
/	3

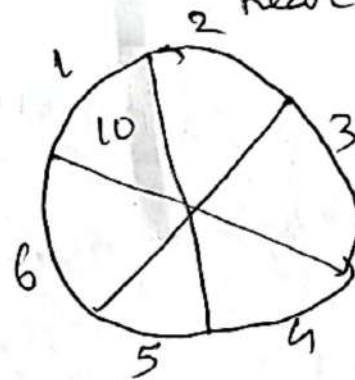
Result = 3

Q.2 Ans

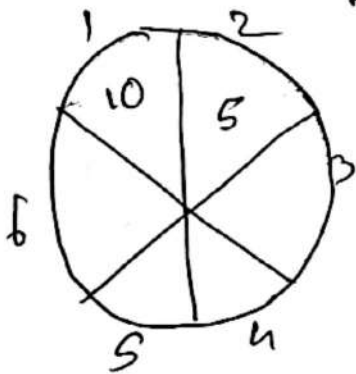
1. Initially, Front = 0
Rear = 0



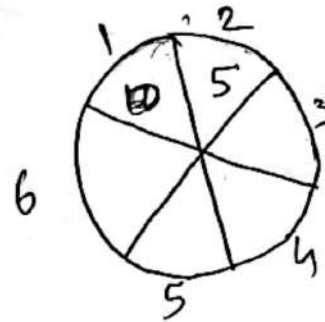
2. Insert 10, Front = 1
Rear = 1



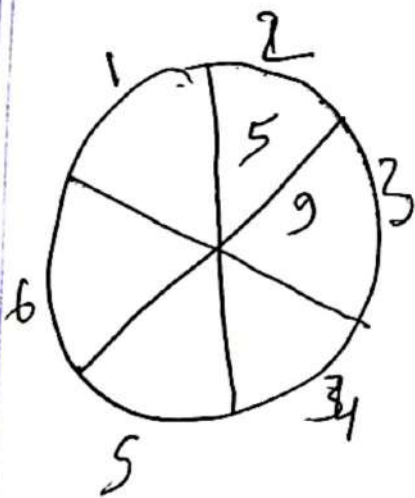
3. Insert 5, Front = 1
Rear = 2



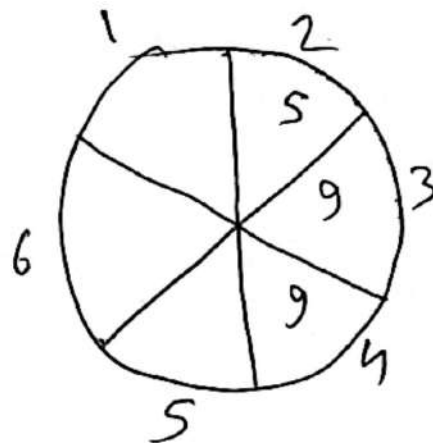
4. Delete(), Front = 2
Rear = 2



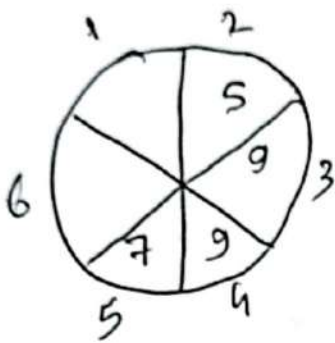
5. Insert 9, Front = 2
Rear = 3



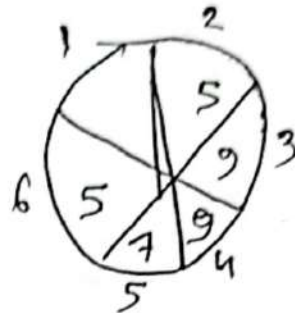
6. Insert 9, Front = 2
Rear = 4



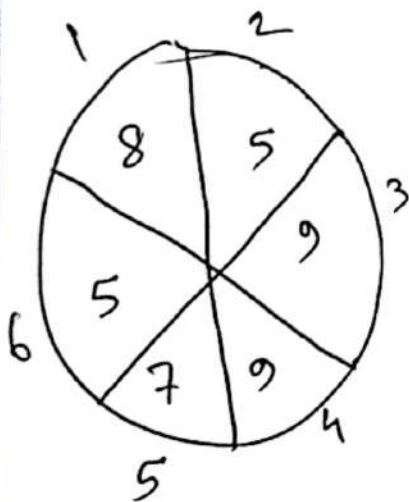
7. Insert 7, Front = 2
Rear = 5



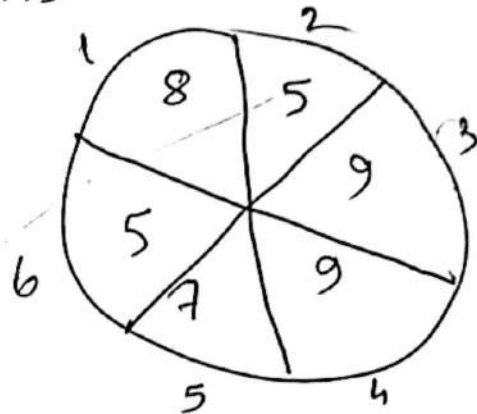
8. Insert 5, Front = 2
Rear = 6



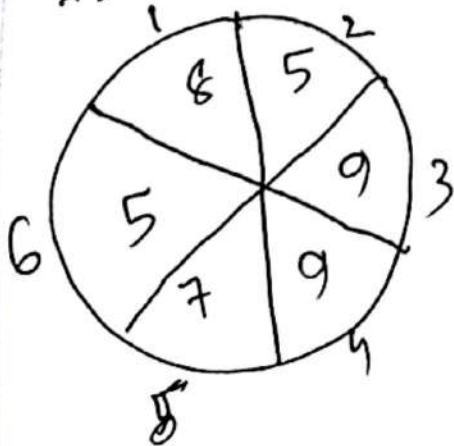
9. Insert 8, Front = 2
Rear = 1



10. Insert 10, Front = 2
Rear = 1
As Front = Rear + 1, Overflow



11. Insert 11, Front = 2, Rear = 1
As Front = Rear + 1, Queue overflow.



Memory Representation of Linked List

Q.3 Ans

- A linked list can be maintained in memory using two linear arrays.
 1. Info
 2. Link
- **Info[K]** represents the information part of a node in the list.
- **Link[K]** represents the next pointer field of a node in the list.
- The beginning of the list is denoted by the variable **Start**.

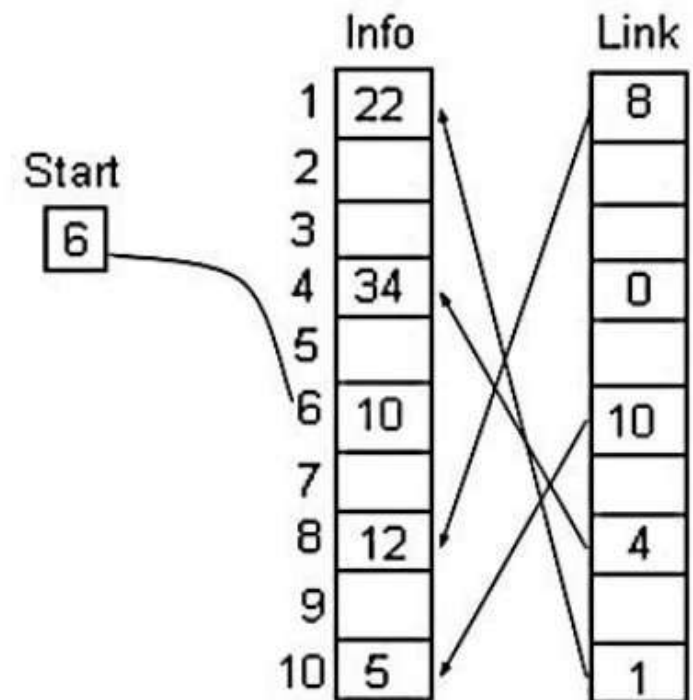
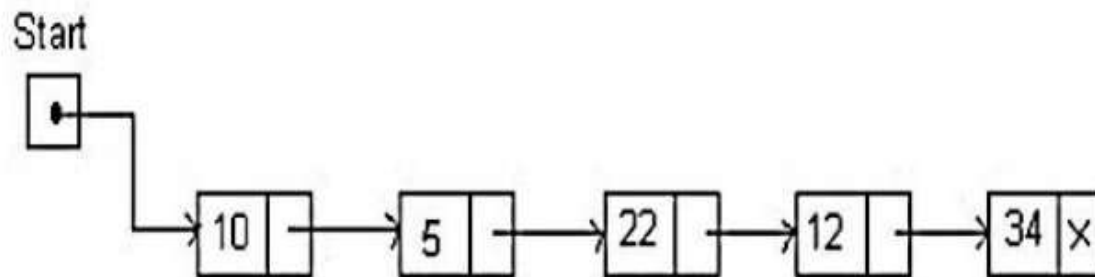


Figure: Linked list & its memory representation

Counting The No. of Elements in a Linked List

Q.4 Ans

Count_Node_List (List, Info, Link, Start)

1. Set PTR := Start and Num := 0
2. While PTR $\neq$ NULL repeat steps 3 and 4
3. Num := Num + 1
4. PTR := Link[PTR]
5. Return.

Q.5
Ans

Singly-linked lists vs. 1D-arrays

ID-array	Singly-linked list
Fixed size: Resizing is expensive	Dynamic size
Insertions and Deletions are inefficient: Elements are usually shifted	Insertions and Deletions are efficient: No shifting
Random access i.e., efficient indexing	No random access □ Not suitable for operations requiring accessing elements by index such as sorting
No memory waste if the array is full or almost full; otherwise may result in much memory waste.	Extra storage needed for references; however uses exactly as much memory as it needs
Sequential access is faster because of greater locality of references [Reason: Elements in contiguous memory locations]	Sequential access is slow because of low locality of references [Reason: Elements not in contiguous memory locations]

Linked Lists versus arrays

Linked lists have several advantages over arrays.

- Elements can be inserted into linked lists indefinitely, while an array will eventually either fill up or need to be resized, an expensive operation that may not even be possible if memory is fragmented.
- An array from which many elements are removed may become wastefully empty or need to be made smaller.

Linked lists have several disadvantages over arrays.

- Arrays allow random access, while linked lists allow only sequential access to the elements.
- Singly-linked lists can only be traversed in one direction. This makes linked lists unsuitable for applications where it is useful to look up an element by its index quickly, such as heapsort.
- Another disadvantage of linked lists is the extra storage needed for references.

Q.6 Ans

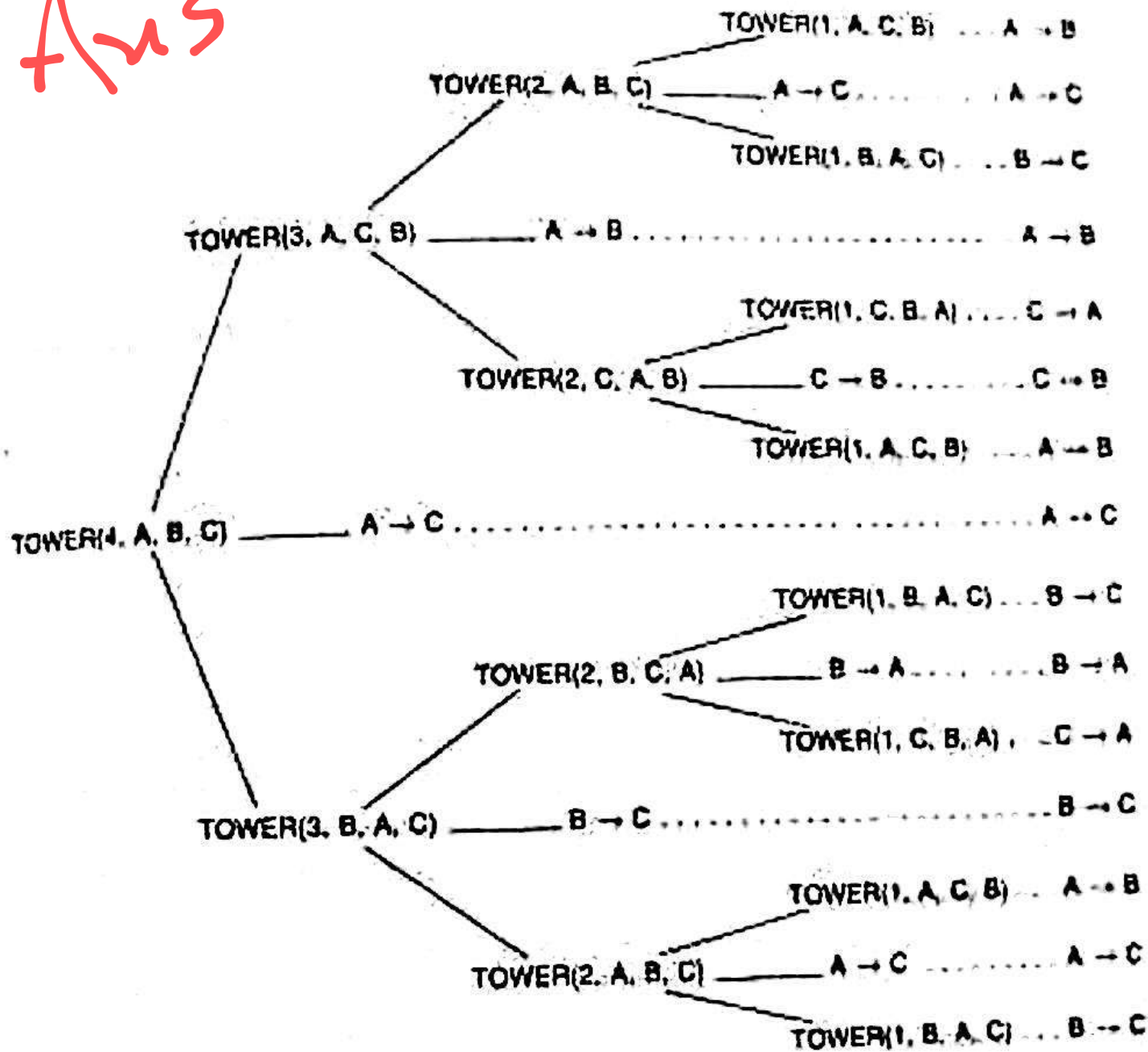


Fig. 6.17 Recursive Solution to Towers of Hanoi Problem for $n = 4$

Stack

Q. 7 Ans

- It is a linear data structure where all additions and deletations are made at one end, the top.
- Stacks are known as LIFO (Last in First out) data structure.
- Stack has only two operation — push and pop
- Two stack errors —

① Underflow : pop an empty stack

② Overflow : push onto an already full stack

• Uses of Stack

① Any sort of nesting

② Evaluating Arithmetic expressions

③ Implementing function or method calls

④ Keeping track of previous choices

Push

① If $Top = MaxSTK$ then Print Overflow and return

② Set $Top := Top + 1$

③ Set $Stack[Top] := Item$

④ Return

Pop

① If $Top = -1$ then print: Underflow and Return.

② set $Item := Stack[Top]$

③ set $top := top - 1$

④ Return.

Convert Infix to postfix $((A+B)*D)^{-1}(E-F)$

Symbol	Stack	Postfix
(	(	
(	((	
(	(((	
A	(((	A
+	(((+	A
B	(((+	AB
)	((	AB +

*	((*	AB +
D	(((*	AB + D
)	((	AB + D *
^	((^	AB + D *
(	(((^	AB + D *
E	(((^	AB + D * E
-	(((^ -	AB + D * E
F	(((^ (-	AB + D * E F
)	(((^	AB + D * E F -
)	((	AB + D * E F - ^

Postfix: $AB + D * E F - ^$

Q.9

Ans

Evaluate the Postfix: 45, 7, 2, -, /, 2, 1, 9, +, *, +

Postfix

Stack

45

45

7

45, 7

2

45, 7, 2

-

45, 5

/

9

2

9, 2

1

9, 2, 1

9

9, 2, 1, 9

+

9, 2, 10

*

9, 20

+

29

Result = 29

Q.10 Ans

Infix to Post. Exp: $k + L - M * N + (O \wedge P) * W / U / V * T + Q$

Symbol	Stack	Postfix
(	(	
k	(	k
+	(+	k
L	(+	k L
-	(-	k L +
M	(-	k L + M
*	(- *	k L + M
N	(- *	k L + M N
+	(+	k L + M N * -
(	(+ (	k L + M N * -
O	(+ (	k L + M N * - O
^	(+ (^	k L + M N * - O
P	(+ (^	k L + M N * - O P
*	(+ (*	k L + M N * - O P ^
W	(+ (* W	k L + M N * - O P ^ W
/	(+ (/	k L + M N * - O P ^ W *
U	(+ (/	k L + M N * - O P ^ W * U
/	(+ (/	k L + M N * - O P ^ W * U /
V	(+ (/	k L + M N * - O P ^ W * U / V
*	(+ (*	k L + M N * - O P ^ W * U / V /
T	(+ (*	k L + M N * - O P ^ W * U / V / T
+	(+ +	k L + M N * - O P ^ W * U / V / T * +
Q	(+	k L + M N * - O P ^ W * U / V / T * + Q
)		k L + M N * - O P ^ W * U / V / T * + Q +

Q.11 Ans

②

ITEM = 110

Ans



10	15	45	56	67	70	80	90	100	110
0	1	2	3	4	5	6	7	8	9
first									last

Int First index = 0

Int last index = 9

Int Middle index = $(0+9)/2 = 4$

Middle value is ~~67~~ 70.

if (Middle value == 110) {
 print (Middle value);

ITEM = 110

0	1	2	3	4	5	6	7	8	9	10
10	15	45	56	67	70	80	90	100	110	120
					first					last

else {
 Now, First index = 5
 last index = 10

Int Middle index = $(5+10)/2$

= 7 so, Middle value = 90

if (Middle value == 110) {
 print (Middle value);

0 1 2 3 4 5 6 7 8 9
 10, 15, 45, 56, 67, 70, 80, 90, 100, 110
 First last
 Now 120
last

First index = 7

last index = 10

$$\text{middle index} = (7 + 10) / 2$$

$$= 8.$$

so, middle value = 100

if (middle value == 100) {
 print (middle value);

→

else,

First index = 8

last index = 10

$$\text{middle index} = (10 + 8) / 2$$

$$= 9.$$

so, middle value = 110

if (middle value == 100) {
 print (middle value);

→ — 2 — =

Two well-defined properties of a recursive procedure are:

1. There must be certain base criteria for which the procedure does not call itself.
2. Each time the procedure does call itself, it must be closer to the base criteria.

• Example:

Factorial Function

- (a) If $n = 0$, then $n! = 1$
- (b) If $n > 0$, then $n! = n \cdot (n-1)!$

Recursive function

Calculate 4!

1. $4! = 4 \cdot 3!$
2. $3! = 3 \cdot 2!$
3. $2! = 2 \cdot 1!$
4. $1! = 1 \cdot 0!$
5. $0! = 1$
6. $1! = 1 \cdot 1 = 1$
7. $2! = 2 \cdot 1 = 2$
8. $3! = 3 \cdot 2 = 6$
9. $4! = 4 \cdot 6 = 24$

Result: $4! = 24$

Fibonacci Sequence [0 1 1 2 3 5 8 13 21 34 55 89.....]

Fibonacci(N)

1. If $N = 0$ or $N = 1$ then Set $Fib := N$ and Return.
2. Set $Fib := Fibonacci(N-2) + Fibonacci(N-1)$
3. Return

Example: Fibonacci(6)

1. $Fib(6) := Fib(4) + Fib(5)$
2. $Fib(6) := Fib(2) + Fib(3) + Fib(5)$
3. $Fib(6) := Fib(0) + Fib(1) + Fib(3) + Fib(5)$
4. $Fib(6) := 0 + 1 + Fib(3) + Fib(5)$
5. $Fib(6) := 1 + Fib(1) + Fib(2) + Fib(5)$
6. $Fib(6) := 1 + 1 + 1 + Fib(3) + Fib(4)$
7. $Fib(6) := 3 + 2 + Fib(4)$
8. $Fib(6) := 5 + Fib(2) + Fib(3)$
9. $Fib(6) := 5 + 1 + 2$
10. $Fib(6) := 8$

Infix $\rightarrow$ Postfix

$$(((A+B) * (C-E)) / (F+G))$$

Practice

Symbol	Stack	Output
	(	
(	((	
(	((	
(	((((	
A	((((	A A
+	((((+	A A
B	((((+	A B
)	((((	AB+
*	(((*	AB+
(	(((*	AB+
C	(((*	AB+C
-	(((*-	AB+C
E	(((*-	AB+CE
)	(((*	AB+CE-
)	((	AB+CE-*
/	((/	AB+CE-*
(	(((/	AB+CE-*
F	(((/	AB+CE-*F
+	(((/+	AB+CE-*F
G	(((/+	AB+CE-*FG
)	((	AB+CE-*FG+
)	(	AB+CE-*FG+/

P.T.O

) | | | AB+CE-*FG+ /

☐ $4*(5+3) - 24/6$

Infix (Q)	stack	Postfix (P)
	(	
4	(4	4
*	(* 4	4
(	(* (4	4
5	(* (4, 5	4, 5
+	(* (+ 4, 5	4, 5
3	(* (+ 4, 5, 3	4, 5, 3
)	(* 4, 5, 3, +	4, 5, 3, +
-	(- 4, 5, 3, +, *	4, 5, 3, +, *
24	(- 4, 5, 3, +, *, 24	4, 5, 3, +, *, 24
/	(- / 4, 5, 3, +, *, 24	4, 5, 3, +, *, 24
6	(- / 4, 5, 3, +, *, 24, 6	4, 5, 3, +, *, 24, 6
)		4, 5, 3, +, *, 24, 6 / -

Postfix calculation : slide

Postfix calculation: 5, 6, 2, +, *, 12, 4, /, -,)

Postfix	Stack
5	5
6	5, 6
2	5, 6, 2
+	5, 8
*	40
12	40, 12
4	40, 12, 4
/	40, 3
-	37
)	37

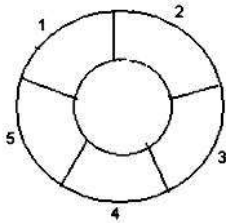
Infix to postfix $\rightarrow ((a+b-c)*d-(e+f))$

(	((	
(	((	
a	((	a
+	((+	a
b	((+	ab
-	((+-	ab+
c	((-	ab+c
)	((	ab+c-
*	((*	ab+c-

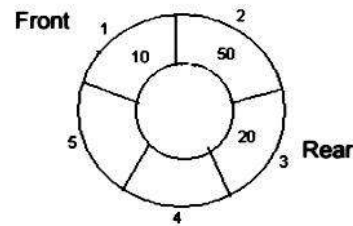
d	((*	ab+c-d
-	((--	ab+c-d*
(	((-(	ab+c-d*
e	((-(	ab+c-d*e
+	((-(+	ab+c-d*e
f	((-(+	ab+c-d*ef
)	((-	ab+c-d*ef+
)	((	ab+c-d*ef+
)		ab+c-d*ef+

Circular queue

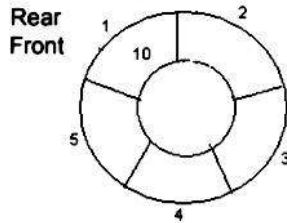
1. Initially, Rear = 0, Front = 0.



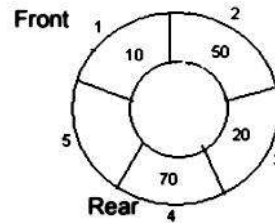
4. Insert 20, Rear = 3, Front = 1.



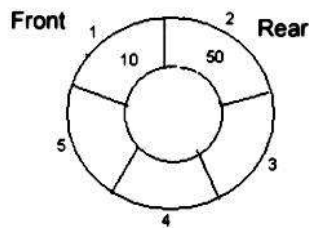
2. Insert 10, Rear = 1, Front = 1.



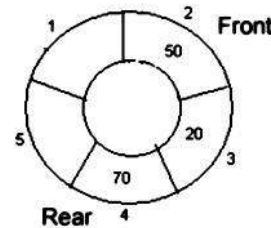
5. Insert 70, Rear = 4, Front = 1.



3. Insert 50, Rear = 2, Front = 1.



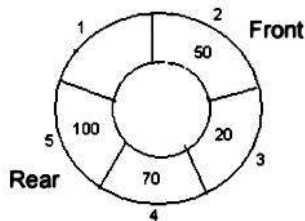
6. Delete front, Rear = 4, Front = 2.



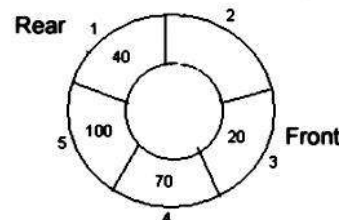
Md. Manowarul Islam, Dept. of CSE, Jagannath University, Dhaka-1100.

Circular queue

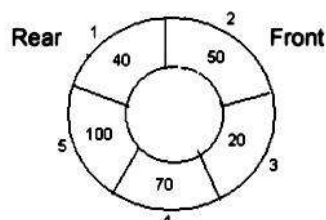
7. Insert 100, Rear = 5, Front = 2.



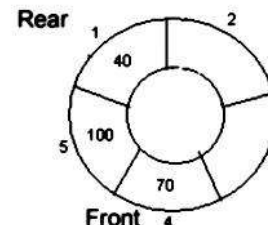
10. Delete front, Rear = 1, Front = 3.



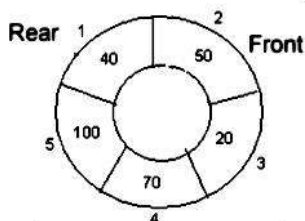
8. Insert 40, Rear = 1, Front = 2.



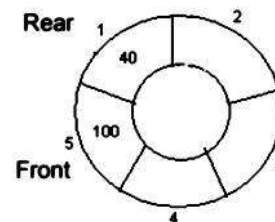
11. Delete front, Rear = 1, Front = 4.



9. Insert 140, Rear = 1, Front = 2.
As Front = Rear + 1, so Queue overflow.



12. Delete front, Rear = 1, Front = 5.



Md. Manowarul Islam, Dept. of CSE, Jagannath University, Dhaka-1100.

Example: Recursive solution to the Towers of Hanoi problem for N=3.

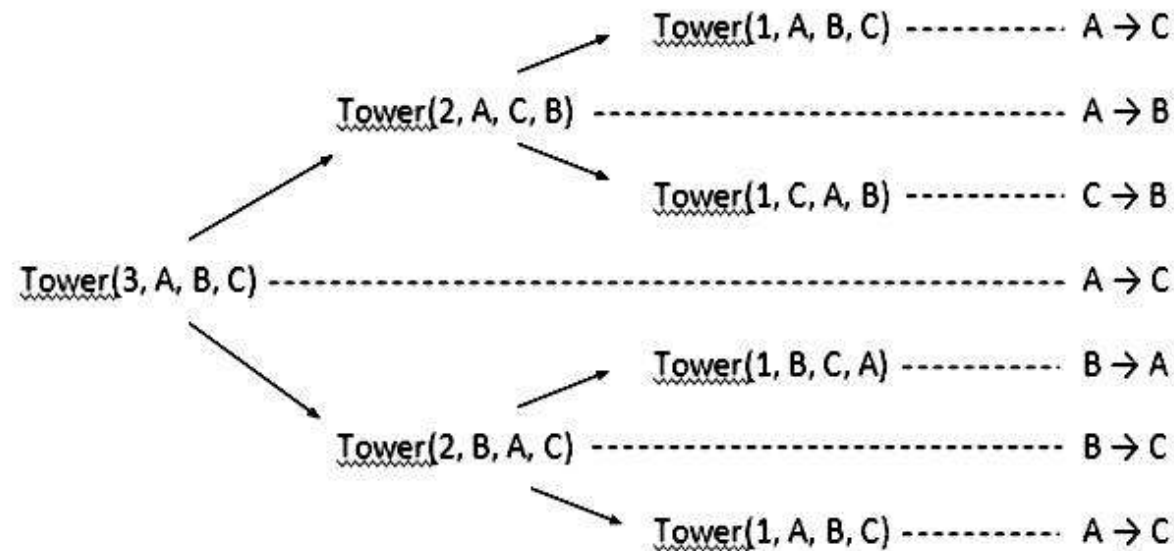


Figure: Recursive solution to the Towers of Hanoi problem for N=3.

Assignment: Write a recursive solution to the Towers of Hanoi for N=5.

East West University

Department of Computer Science and Engineering

Course Code: CSE 207 Title: Data Structures

Course Instructor: Manowarul Islam

Date: 30.07.2025 Summer 2025

Mid-Term Examination Section: 11

Marks:30

Time: 1 hour 20 minutes

Note: Answer all questions. Figure in the right margin indicate marks.

1 Write a simple procedure to insert an element into a circular queue. CO1-4

X What is stack? Convert the following expression to equivalent postfix expression using stack: $K + L - M * N + (O \wedge P) * W / U / V * T + Q$ CO2-8

X You are given the following **sorted array** of integers: $A = [5, 9, 12, 17, 23, 28, 34, 39, 45, 51, 60, 72]$. What is the index of 70 when using the ternary search algorithm? CO1-6 4

X How linked list is more efficient for insertion and deletion operation compare to array. CO2-5

OR

What are the properties of recursive function? Write the solution for Tower of Hanoi problem for $N=4$. CO2-5

X What is the problem of linear queue? What is queue? Suppose you have a circular queue of size 6 (1 to 6). Initially the value of the front and rear is 0. Now show the state of the queue if the following operation is performed on the queue. Also find the value of front and rear in each step. CO1-7

~~insert (10); insert (5), delete(), delete(), insert (9); insert (9); insert (7);~~
~~insert (5); insert (8); insert (10); insert (11).~~

East West University
Department of Computer Science and Engineering
Course Code: CSE 207 Title: Data Structures
Course Instructor: Manowarul Islam
Date: 30.07.2025 Summer 2025
Mid-Term Examination Section: 10
Marks: 30

Time: 1 hour 20 minutes

Note: Answer all questions. Figure in the right margin indicate marks.

- 1 Write a simple procedure to delete an element into a circular queue. CO1-4
- 2 Write an algorithm to evaluate a postfix expression. Evaluate the following postfix expression using your algorithm. CO2-7
3, 5, +, 6, 4, -, *, 4, 1, -, 2, ^, +
- 3 What are the properties of binary search algorithm? You are given the following **sorted array** of integers: A = [5, 9, 12, 17, 23, 28, 34, 39, 45, 51, 60, 72]. What is the index of 72 when using the ternary search algorithm? CO1-7
- 4 What are the advantages of using linked list over an array? Justify your answer. CO2-5

OR

What are the properties of recursive function? Write a **recursive function** that calculates the sum of all integers from 1 to N. CO2-5

- The function should take a single integer N as input.
- It should return the sum of all numbers from 1 up to N.
- If N is 0, the function should return 0.

- 5 What is the problem of linear queue? Suppose you have a circular queue of size 7 (1 to 7). Initially the value of the front and rear is 0. Now show the state of the queue if the following operation is performed on the queue. Also find the value of front and rear in each step. CO1-7

delete(), delete(); insert(10); insert(15); insert(9); insert(7); insert(17); delete(); insert(8); delete(); insert(11); insert(13); insert(11).

East West University
Department of Computer Science and Engineering
Course Code: CSE 207 Title: Data Structures
Course Instructor: Manowarul Islam
Date: 03.03.2022 Fall 2022
ClassTest-1 Section: 5
Time: 20 minutes Marks:10

Note: Answer all questions. Figure in the right margin indicate marks.

- | | | |
|---|--|---|
| 1 | Define stack. Write down the basic steps for PUSH operation. | 3 |
| 2 | Convert the following expression to postfix
Q: $((A+B)*D) \uparrow (E-F)$. | 4 |
| 3 | Evaluate the following Postfix expression. P=45, 7, 2, -, /, 2, 1, 9, +, *, + | 3 |